Voxel Car / SynthOccPred: Technical Report

Contents

1 Executive Summary

The repository implements a synthetic autonomous-driving research stack built around four tightly coupled components:

1. a *procedural voxel world generator* that creates drivable terrain and trajectories;
2. a *camera renderer* that ray-marches first-person images from the voxel world;
3. a *monocular occupancy predictor* (`OccNet`) that maps a single RGB image into an ego-frame 3D occupancy tensor;
4. a *closed-loop control stack* that either uses A* over predicted occupancy or a PPO policy to drive the simulated vehicle.

The project is intentionally structured as an end-to-end perception–planning–control pipeline with a synthetic data engine. The central scientific idea is to learn an ego-centric occupancy representation from monocular imagery in a fixed-camera setting, then evaluate whether that representation is sufficient for downstream closed-loop navigation.

At a high level, the complete chain is

procedural world → rendered camera image → ego occupancy prediction → planner / RL policy → simulator

The codebase also contains auxiliary modules for state estimation (Kalman filtering and EKF), a Gradio demonstration interface, Hugging Face Hub integration, preprocessing utilities for large synthetic datasets, and artifact generation for RL training.

2 System-Level Architecture

The source package is organized under `src/voxel_car`. Conceptually, the modules form the following dependency graph:

- **common**: shared dataclasses and constants (camera, world, trajectory, rendering configs; noise utilities);
- **worldgen**: random composite trajectories, terrain generation, evaluation scenarios;
- **rendering**: camera projection, ray-marched first-person views, bird's-eye visualization, composed video frames;
- **geometry**: ego-frame grid construction, field-of-view masking, world-to-ego voxel sampling;
- **datasets**: synthetic sample generation and memmapped training-dataset preparation;
- **perception**: occupancy network architecture, training visualizations;
- **planning**: occupancy-to-BEV conversion, inflated-cost A*, collision checks, local steering logic;
- **simulation**: closed-loop visual rollout engine and visualization;
- **rl**: fast oracle-BEV Gymnasium environment, PPO adapter, training artifacts;

- **estimation**: linear Kalman filter, EKF, synthetic sensor models;
- **ui**: Gradio interface that exposes both data generation and closed-loop demonstrations;
- **hub**: utilities for resolving models from local disk or Hugging Face Hub.

The design is not merely a set of scripts. It encodes a clear modeling assumption:

> A fixed monocular camera, coupled to a fixed ego-grid definition, should permit learning a mapping from image appearance to near-field 3D occupancy sufficient for local planning.

That assumption drives the geometry, data pipeline, masking strategy, evaluation setup, and RL interface.

3 Coordinate Frames and Conventions

A substantial fraction of the repository is devoted to keeping coordinate conventions consistent. This is one of the most important technical details in the system.

3.1 World Frame

The world frame is right-handed with

$$X = \text{east}, \qquad Y = \text{north}, \qquad Z = \text{up}.$$

A voxel indexed by (i, j, k) occupies the half-open cube

$$[i, i+1) \times [j, j+1) \times [k, k+1).$$

Thus world occupancy is a Boolean tensor

$$\mathcal{V} \in \{0, 1\}^{G_x \times G_y \times G_z}.$$

3.2 Vehicle Frame

The vehicle is represented in 2D by a heading vector

$$\mathbf{h} = (h_x, h_y), \qquad \|\mathbf{h}\|_2 = 1.$$

The car's rightward direction is defined as

$$\mathbf{r} = (h_y, -h_x).$$

This convention is used consistently in world-to-ego transforms, camera placement, and collision footprint calculations.

3.3 Ego Occupancy Frame

The learned occupancy tensor is expressed in an ego-aligned 3D grid:

$$\text{ego}_x = \text{forward}, \qquad \text{ego}_y = \text{right}, \qquad \text{ego}_z = \text{up}.$$

The default ego grid configuration is

$$D_x = 20, \qquad D_y = 16, \qquad D_z = 12, \qquad \Delta = 1 \text{ m / cell}.$$

The center of ego cell (i, j, k) is

$$\mathbf{c}_{ijk} = \Big((i + \tfrac{1}{2})\Delta,\ (j + \tfrac{1}{2} - \tfrac{D_y}{2})\Delta,\ (k + \tfrac{1}{2})\Delta\Big).$$

Hence the grid spans the near-field volume

$$\text{ego}_x \in [0, D_x\Delta), \qquad \text{ego}_y \in \Big[-\frac{D_y}{2}\Delta, \frac{D_y}{2}\Delta\Big), \qquad \text{ego}_z \in [0, D_z\Delta).$$

A key modeling choice is that the grid begins at the vehicle center in the forward direction rather than extending behind the car. This aligns the learned representation with forward-driving tasks.

3.4 Camera Frame

The renderer uses the OpenCV camera convention:

$$X_{\text{cam}} = \text{right}, \qquad Y_{\text{cam}} = \text{down}, \qquad Z_{\text{cam}} = \text{forward}.$$

Camera yaw is defined relative to vehicle forward such that positive yaw rotates toward the car's right side. Therefore:

yaw $= 0^\circ \Rightarrow$ forward camera, yaw $= 90^\circ \Rightarrow$ right-facing camera, yaw $= -90^\circ \Rightarrow$ left-facing camera, yaw =

4 Procedural World Generation

The world generator is composed from two independent stochastic objects:

(a) a random reference trajectory, and

(b) a terrain height field shaped around that trajectory.

This separation is critical. The trajectory is sampled first; the terrain is then carved around the trajectory to guarantee a drivable corridor.

4.1 Composite Trajectory Model

The trajectory generator in `worldgen/trajectory.py` builds a path by concatenating random segments chosen from

$$\{\texttt{line},\ \texttt{arc},\ \texttt{sine}\}.$$

Let the current state be position $\mathbf{p} = (x, y)$ and heading angle θ.

4.1.1 Line Segment

A line segment of length L is

$$\mathbf{p}(t) = \mathbf{p}_0 + t(\cos\theta, \sin\theta), \qquad t \in [0, L].$$

The final heading is unchanged:

$$\theta' = \theta.$$

4.1.2 Circular Arc Segment

An arc is parameterized by signed radius R and arc length L. The total angular sweep is

$$\phi = \frac{L}{R}.$$

The implementation uses the exact circular-arc displacement formula

$$\Delta x = R\big(\sin(\theta + \phi) - \sin\theta\big), \qquad \Delta y = R\big(-\cos(\theta + \phi) + \cos\theta\big),$$

with updated heading

$$\theta' = \theta + \phi.$$

A sweep cap of roughly 0.9π radians is enforced to prevent the trajectory from doubling back too aggressively.

4.1.3 Sine-Wiggle Segment

The sine segment is not a sinusoid in the world frame; it is a forward-moving segment with perpendicular oscillation. Let

$$\mathbf{f} = (\cos\theta, \sin\theta), \qquad \mathbf{p}_\perp = (-\sin\theta, \cos\theta)$$

be forward and left-perpendicular directions. For arc-length parameter $t \in [0, L]$, the normalized progress is

$$\tau = \frac{t}{L}.$$

The lateral offset is

$$\omega(\tau) = A\,\sin^2(\pi\tau)\,\sin(2\pi C\tau),$$

where A is amplitude and C is the number of cycles.

The resulting path is

$$\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{f} + \omega(\tau)\mathbf{p}_\perp.$$

The envelope $\sin^2(\pi\tau)$ is a subtle and technically important choice. It forces both offset and offset derivative to be zero at segment boundaries, which preserves tangent continuity across segment joins.

4.2 Trajectory Noise and Smoothing

After segment composition, Gaussian waypoint noise is added:

$$\widetilde{\mathbf{p}}_i = \mathbf{p}_i + \boldsymbol{\epsilon}_i, \qquad \boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \sigma^2 I).$$

Each coordinate then passes through a box smoother of window size w:

$$\widehat{x}_i = \frac{1}{w} \sum_{k=i-m}^{i+m} \widetilde{x}_k, \qquad \widehat{y}_i = \frac{1}{w} \sum_{k=i-m}^{i+m} \widetilde{y}_k,$$

where $m = \lfloor w/2 \rfloor$. Boundary samples are preserved explicitly to avoid distorting the entry/exit endpoints.

Finally, points are clipped to remain inside a safety margin from the world boundary:

$$\widehat{x}_i \in [m, G_x - m - 1], \qquad \widehat{y}_i \in [m, G_y - m - 1].$$

4.3 Multi-Octave Value Noise Terrain

Terrain is synthesized by multi-octave value noise. For octave o, a coarse lattice of random values is generated and bilinearly interpolated to full resolution. The interpolation uses the smoothstep polynomial

$$S(t) = t^2(3 - 2t),$$

which guarantees zero first derivative at cell boundaries.

If $L_o(x, y)$ is the upsampled field at octave o, then the final noise is

$$N(x, y) = \frac{1}{\sum_o a_o} \sum_o a_o L_o(x, y), \qquad a_{o+1} = p a_o,$$

with persistence $p = 0.5$ by default and octave frequency doubling.

The terrain height is shaped nonlinearly as

$$H(x, y) = (N(x, y))^{1.8}\, H_{\max},$$

then thresholded so that very low-noise regions are flattened:

$$N(x, y) < 0.45 \Longrightarrow H(x, y) = 0.$$

The exponent 1.8 suppresses mid-range elevations and produces broader lowlands with more concentrated tall structures.

4.4 Trajectory-Aware Road Carving

The trajectory is used to carve a guaranteed drivable corridor into the terrain. For each waypoint, a local patch is examined. Let d be the Euclidean distance from a terrain cell to the nearest trajectory point under the local approximation.

If r is the strict road radius and $R_{\text{sh}} = r + s$ is the road-plus-shoulder radius, then the carving rule is:

$$H'(x,y) = \begin{cases} 0, & d \le r, \\ \min\left(H(x,y),\ H(x,y)\dfrac{d-r}{R_{\text{sh}}-r}\right), & r < d \le R_{\text{sh}}, \\ H(x,y), & d > R_{\text{sh}}. \end{cases}$$

This is an important modeling improvement over hard clipping. The road is guaranteed flat, while the shoulder provides a smooth ramp from road to obstacle field rather than a vertical wall. In planning terms, this creates traversable corridors that still encode a geometric notion of roadside risk.

4.5 Voxel Lifting

Given the integer height field $H'(x,y)$, the Boolean voxel grid is defined by

$$\mathcal{V}(i,j,k) = \mathbf{1}\{k \le H'(i,j)\}.$$

The bottom layer is forcibly occupied:

$$\mathcal{V}(i,j,0) = 1 \quad \forall i,j.$$

This makes the ground opaque to ray marching and simplifies collision logic. It also implies that all BEV obstacle logic must ignore $z = 0$ to avoid treating the ground as a wall.

5 Scenario Generation for Closed-Loop Evaluation

Evaluation scenarios are generated by a dedicated preset system in `worldgen/scenarios.py`. Each preset defines a distribution over terrain roughness, road width, obstacle height, and path complexity.

A scenario consists of:

- a voxel world $\mathcal{V}$,
- a reference trajectory $\{\mathbf{p}_t\}_{t=0}^{T-1}$,
- an entry point $\mathbf{p}_0$,
- an exit point $\mathbf{p}_{T-1}$,
- an initial heading derived from the first few waypoints.

The initial heading is computed by averaging the early path direction:

$$\mathbf{h}_0 = \frac{\mathbf{p}_\ell - \mathbf{p}_0}{\|\mathbf{p}_\ell - \mathbf{p}_0\|_2}, \qquad \ell = \min(3, T-1).$$

This reduces sensitivity to local waypoint noise and prevents the vehicle from starting with a spurious orientation.

6 Camera Geometry and First-Person Rendering

6.1 Camera Pose Relative to Vehicle

For a camera with forward offset f, right offset r, height h, and yaw ψ, the camera world position is

$$\mathbf{c} = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} x_{\text{car}} \\ y_{\text{car}} \\ 0 \end{bmatrix} + f \begin{bmatrix} h_x \\ h_y \\ 0 \end{bmatrix} + r \begin{bmatrix} h_y \\ -h_x \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ h \end{bmatrix}.$$

The camera's forward heading in the world plane is

$$\mathbf{h}_{\text{cam}} = \cos\psi\, \mathbf{h} + \sin\psi\, \mathbf{r}.$$

The corresponding OpenCV camera axes in world coordinates are:

$$\text{forward} = \mathbf{h}_{\text{cam}}, \qquad \text{right} = (h_{\text{cam},y}, -h_{\text{cam},x}, 0), \qquad \text{down} = (0, 0, -1).$$

The camera-to-world rotation matrix is assembled columnwise as

$$R_{\text{cam}\to\text{world}} = [\mathbf{r}_{\text{cam}}\ \mathbf{d}_{\text{cam}}\ \mathbf{f}_{\text{cam}}].$$

6.2 Pinhole Ray Construction

For image width W, height H, and horizontal FOV φ_h, the focal length in pixel units is

$$f_x = \frac{0.5W}{\tan(\varphi_h/2)}, \qquad f_y = f_x.$$

The code assumes square pixels. With principal point

$$c_x = \frac{W-1}{2}, \qquad c_y = \frac{H-1}{2},$$

per-pixel normalized camera-frame rays are

$$\mathbf{d}_{\text{cam}}(u, v) \propto \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}.$$

These are normalized and rotated into world coordinates via

$$\mathbf{d}_{\text{world}} = R_{\text{cam}\to\text{world}}\, \mathbf{d}_{\text{cam}}.$$

6.3 Ray Marching

For each pixel, the renderer samples depths

$$t_1, \ldots, t_N \in [t_{\text{near}}, t_{\text{far}}]$$

and evaluates world points

$$\mathbf{x}(t_n) = \mathbf{c} + t_n \mathbf{d}_{\text{world}}.$$

The first occupied voxel intersected by the sample sequence is treated as the visible surface. This is a simple first-hit discrete ray marcher, not a full volumetric renderer.

Shading is derived from a crude estimate of the surface normal based on the difference between the hit voxel and the previous sample voxel. A Lambertian term is applied using a fixed light vector, then blended with a sky color using a depth-based fog term.

This renderer is intentionally approximate. Its role is to produce varied but structured monocular images, not photorealistic imagery.

7 Bird's-Eye Rendering and Visualization

The BEV renderer projects the voxel world into a top-down RGB map by taking the highest occupied voxel in each (x, y) column. Terrain colors are assigned as a function of top height, with a distinct palette for ground and obstacles.

The renderer overlays:

- the reference trajectory,
- the actual driven trajectory,
- camera frustum wedges,
- a rotated rectangle vehicle glyph,
- a heading indicator,
- entry and exit markers.

This is not used by the control stack directly; it is for diagnostics and artifact generation. However, the visualizations expose the planner's behavior and make failure modes legible.

8 Ego-Frame Geometry and Ground-Truth Occupancy Extraction

8.1 World-to-Ego Coordinate Transform

For a world point $\mathbf{w} = (x_w, y_w)$ and car position $\mathbf{p} = (x_c, y_c)$ with heading $\mathbf{h} = (h_x, h_y)$, the ego-frame coordinates are

$$e_x = (x_w - x_c)h_x + (y_w - y_c)h_y,$$
$$e_y = (x_w - x_c)h_y - (y_w - y_c)h_x.$$

The inverse transform is

$$x_w = x_c + e_x h_x + e_y h_y,$$
$$y_w = y_c + e_x h_y - e_y h_x.$$

These formulas follow directly from the chosen right-vector convention and are used in planning, RL observations, and occupancy sampling.

8.2 Ego Voxel Ground Truth

Each ego voxel center $\mathbf{c}_{ijk}$ is mapped into world space using the current car pose and heading:

$$x_w = x_c + e_x h_x + e_y r_x,$$
$$y_w = y_c + e_x h_y + e_y r_y,$$
$$z_w = e_z.$$

The world coordinates are floored to voxel indices and sampled from the world occupancy tensor. Out-of-bounds lookups return false.

Thus the ego occupancy tensor is simply a rigidly transformed crop of the world voxel field:

$$\mathcal{O}_{\text{ego}}(i,j,k) = \mathcal{V}(\lfloor x_w \rfloor, \lfloor y_w \rfloor, \lfloor z_w \rfloor)$$

where defined.

9 Field-of-View Masking

9.1 Why a Static Mask Exists

Because both the camera and the ego grid are rigidly attached to the car, the set of ego voxels visible from the camera is invariant across samples. This is one of the most important simplifying assumptions in the entire pipeline.

A single Boolean mask

$$M \in \{0,1\}^{D_x \times D_y \times D_z}$$

can therefore be computed once and reused for all training examples.

9.2 Mask Construction

Let a voxel center in ego coordinates be $\mathbf{q} = (e_x, e_y, e_z)$ and camera position in ego coordinates be

$$\mathbf{c}_{\text{ego}} = (f, r, h).$$

If the camera yaw is ψ, its forward and right vectors in ego coordinates are

$$\mathbf{f}_{\text{cam}} = (\cos\psi, \sin\psi, 0), \qquad \mathbf{r}_{\text{cam}} = (-\sin\psi, \cos\psi, 0).$$

The point relative to the camera is

$$\mathbf{v} = \mathbf{q} - \mathbf{c}_{\text{ego}}.$$

The corresponding camera-frame coordinates are

$$z_{\text{cam}} = \mathbf{v} \cdot \mathbf{f}_{\text{cam}}, \qquad x_{\text{cam}} = \mathbf{v} \cdot \mathbf{r}_{\text{cam}}, \qquad y_{\text{cam}} = -v_z.$$

If the horizontal FOV is φ_h and the vertical FOV is derived from aspect ratio,

$$\varphi_v = 2 \arctan\left(\frac{H}{W} \tan\frac{\varphi_h}{2}\right),$$

then with angular slack δ the inclusion rule is

$$M(i,j,k) = 1 \iff \begin{cases} z_{\text{cam}} > 0, \\ \left|\dfrac{x_{\text{cam}}}{z_{\text{cam}}}\right| < \tan\left(\dfrac{\varphi_h}{2} + \delta\right), \\ \left|\dfrac{y_{\text{cam}}}{z_{\text{cam}}}\right| < \tan\left(\dfrac{\varphi_v}{2} + \delta\right). \end{cases}$$

The angular margin defaults to roughly 3°. This creates a slightly broader supervision region than the strict optical frustum and reduces edge artifacts.

9.3 Learning Consequence

The network is penalized only on masked voxels. Unmasked voxels do not contribute to the loss. That means the model is not trained to infer occupancy everywhere in the ego grid; it is trained only where occupancy is considered geometrically observable from the fixed camera.

This is a sensible choice, but it creates an important downstream issue: voxels very close to the vehicle may fall below the camera's vertical FOV and therefore be unsupervised. The planner includes explicit logic to guard against this blind spot.

10 Dataset Generation and Preprocessing

10.1 Synthetic Run Contents

A generated run contains:

- the voxel world,
- the reference trajectory,
- a BEV snapshot,
- a BEV video,
- one first-person video per enabled camera,
- a metadata file encoding all generation parameters and file paths.

The data format is deliberately self-describing so downstream tools can reconstruct camera configuration, render geometry, and frame indexing from metadata alone.

10.2 Training Motivation for Preprocessing

Training from compressed video files is inefficient because frame-seeking in MP4 is expensive and repeated decompression wastes CPU time. The preprocessing stage decodes each frame once, computes the ego occupancy target once, and stores both in flat contiguous arrays.

The preprocessed dataset consists of

$$\begin{aligned} \texttt{images.uint8.npy} &\in \{0,\dots,255\}^{N\times 3\times H\times W},\\ \texttt{gt.uint8.npy} &\in \{0,1\}^{N\times D_x\times D_y\times D_z},\\ \texttt{mask.uint8.npy} &\in \{0,1\}^{D_x\times D_y\times D_z}. \end{aligned}$$

The corresponding index file stores sample-to-run mapping and trajectory indices. This is what enables run-level train/validation/test splits.

10.3 Run-Level Splitting

The dataset split logic is based on generated runs rather than frames. This is critical. Frames from the same run share world geometry and are highly correlated. A frame-level split would leak world structure from training into validation and test, inflating performance.

If $\mathcal{R}$ is the set of run IDs and each sample s has run assignment $r(s)$, then the split is a partition of $\mathcal{R}$, and samples inherit the split of their run.

11 Monocular Occupancy Predictor (OccNet)

11.1 Architecture

The network maps an RGB image

$$\mathbf{I} \in [0,1]^{3\times H\times W}$$

into occupancy logits

$$\mathbf{Z} \in \mathbb{R}^{D_x\times D_y\times D_z}.$$

The architecture is intentionally lightweight:

1. a 2D CNN encoder with several stride-2 convolution blocks;
2. bilinear interpolation of the feature map to spatial size (D_x, D_y);
3. a BEV head producing D_z channels;
4. a permutation from (D_z, D_x, D_y) to (D_x, D_y, D_z).

The model therefore performs a learned image-to-BEV lift without explicit camera projection. Mathematically, if the encoder outputs

$$\mathbf{F}_{\text{enc}} \in \mathbb{R}^{C\times h'\times w'},$$

then bilinear resize yields

$$\mathbf{F}_{\text{bev}} \in \mathbb{R}^{C\times D_x\times D_y}.$$

The head produces

$$\mathbf{U} \in \mathbb{R}^{D_z \times D_x \times D_y},$$

and the final output is

$$\mathbf{Z}(i, j, k) = \mathbf{U}(k, i, j).$$

11.2 Interpretation

This is not a geometrically explicit inverse-perspective mapping. It is a proof-of-concept architecture that relies on fixed camera placement and synthetic data regularity to let the CNN learn an implicit monocular depth-and-occupancy prior.

The architecture trades physical interpretability for simplicity and speed. It is computationally cheap compared with more sophisticated view-transformer designs, but it also inherits limitations:

- no explicit multi-view geometry,
- no uncertainty representation,
- no temporal fusion,
- no explicit camera intrinsics/extrinsics layer,
- coarse occupancy discretization.

12 Loss Function and Metrics

12.1 Masked BCE with Positive Weighting

Let Z_{bijk} be the output logit for batch element b and voxel (i, j, k), and let $Y_{bijk} \in \{0, 1\}$ be the target occupancy. Let $M_{ijk} \in \{0, 1\}$ be the static FOV mask. The code uses voxelwise binary cross entropy with logits and positive-class weight $\lambda > 0$:

$$\ell(z, y) = -\lambda y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)).$$

The masked batch loss is

$$\mathcal{L} = \frac{1}{B \sum_{ijk} M_{ijk} + \varepsilon} \sum_{b=1}^{B} \sum_{i,j,k} M_{ijk}\, \ell(Z_{bijk}, Y_{bijk}).$$

This choice reflects two realities:

(a) only visible voxels should be supervised;

(b) occupied voxels are sparse relative to empty voxels, so class imbalance must be countered.

12.2 Thresholding Convention

Predictions are thresholded at logit zero:

$$\widehat{Y}_{bijk} = \mathbf{1}\{Z_{bijk} > 0\}.$$

This is equivalent to thresholding the sigmoid at 0.5.

12.3 Metrics

Within the mask, the code accumulates true positives, false positives, false negatives, and true negatives. Occupied-class IoU is

$$\mathrm{IoU} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}.$$

Additional metrics are

$$\mathrm{Accuracy} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN} + \mathrm{TN}},$$

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad \mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}.$$

Because the target is extremely sparse, IoU is the most meaningful primary metric here. Accuracy is much less informative because true negatives dominate.

13 Perception Visualization

The visualization code collapses the 3D occupancy tensors along height:

$$\mathrm{BEV}(i, j) = \bigvee_{k} \mathcal{O}(i, j, k).$$

Prediction and ground truth are then compared cellwise under the BEV-collapsed mask. The resulting categories are

- TP: correct obstacle prediction,
- FN: missed obstacle,
- FP: hallucinated obstacle,
- TN: correct empty,
- out-of-mask: unsupervised cell.

This is useful because downstream planning is also performed in a BEV reduction of the 3D occupancy, so BEV error maps are closer to planning relevance than full 3D voxel slices.

14 Closed-Loop Planning with Predicted Occupancy

14.1 Occupancy-to-BEV Obstacle Map

The planner first collapses predicted occupancy above ground level into a 2D obstacle map:

$$\mathcal{B}(i, j) = \bigvee_{k \geq z_{\min}} \widehat{Y}(i, j, k), \qquad z_{\min} = 1.$$

Ground layer $z = 0$ is ignored because it is always occupied by construction and would otherwise mark the entire BEV as blocked.

14.2 Blind-Spot Handling

A central planning subtlety is the close-range blind spot. Some near-car voxels lie below the camera's vertical FOV and are therefore unsupervised. If the planner naively masked predictions by visibility, it would falsely mark these regions free.

The implementation handles this by:

1. using masked predictions in general,
2. but allowing raw predictions in the close-range rows,
3. optionally marking unsupervised close-range cells as obstacles by default,
4. then carving a narrow centerline corridor so the car can still move forward if those cells are not explicitly predicted occupied.

This is a conservative heuristic rather than a probabilistic visibility model, but it directly addresses a failure mode that would otherwise produce frequent collisions.

14.3 Obstacle Inflation

Given a BEV obstacle mask, morphological dilation is applied to create a vehicle safety buffer. If $\mathcal{B}$ is the original obstacle set, the inflated obstacle set is roughly

$$\mathcal{B}_{\text{infl}} = \mathcal{B} \oplus S_r,$$

where S_r is a small discrete structuring element induced by repeated four-connected dilation. With 1 meter cells and default inflation radius 2, the buffer is approximately the half-width safety margin needed for a 2.4 meter vehicle.

14.4 Distance-Transform Soft Cost

The planner augments the hard obstacle map with a soft cost based on distance to obstacles. Let $d(i, j)$ be the Chebyshev distance transform of the inflated obstacle map. Then the soft cost is

$$C_{\text{soft}}(i, j) = w\, e^{-d(i,j)},$$

where w is a tunable weight.

This has two effects:

- cells adjacent to obstacles have a noticeable penalty,
- cells deep in free space have near-zero penalty.

Consequently, A* tends to prefer corridor centers rather than grazing obstacle boundaries.

14.5 A* Search State and Costs

The planner uses 8-connected A* on the BEV grid. The state includes not just the cell (i, j) but also the incoming motion direction $(\Delta i, \Delta j)$ so that turning can be penalized.

If a move from cell c to c' has geometric step cost $\|c' - c\|$, soft cost $C_{\text{soft}}(c')$, and incoming direction change penalty based on cosine similarity, then the transition cost is approximately

$$\text{cost}(c \to c') = \|c' - c\| + C_{\text{soft}}(c') + \beta(1 - \cos\alpha),$$

where α is the angle between consecutive move directions and β is the heading penalty coefficient.

This creates smoother plans than vanilla A* without requiring a full kinodynamic search.

14.6 Subgoal Selection

The planner does not directly aim A* at the global goal projected into the local ego grid. Instead it chooses a reachable *subgoal* that balances forward progress and lateral alignment with the global goal.

Let (g_x, g_y) be the world goal in ego coordinates and let (b_x, b_y) be a blended target direction:

$$\tilde{b}_x = \eta \cdot 1 + (1 - \eta)\frac{g_x}{\sqrt{g_x^2 + g_y^2}},$$
$$\tilde{b}_y = (1 - \eta)\frac{g_y}{\sqrt{g_x^2 + g_y^2}},$$

with forward-bias parameter $\eta \in [0, 1]$, then normalized to $\mathbf{b} = (b_x, b_y)$. For each forward row i, the desired column is approximated by the ray intersection

$$j_{\text{des}} \approx j_0 + (i - i_0)\frac{b_y}{b_x}.$$

The code searches for the nearest free cell in that row and scores candidates by

$$\text{score}(i, j) = i - 0.5\,|j - j_{\text{des}}|.$$

The best-scoring forward cell becomes the local A* goal.

This is a practical hybrid between global-goal tracking and local frontier following.

14.7 Goal Slowdown

Near the goal, the planner reduces step size linearly to avoid overshooting. If the current world distance to goal is d and the slow radius is R, the effective step fraction is

$$\rho(d) = \max\left(\rho_{\min}, \frac{d}{R}\right), \qquad d < R,$$

and equals 1 outside the radius. Thus

$$s_{\text{eff}} = s\,\rho(d).$$

14.8 Fallback Behavior

If A* fails on the inflated cost map, the planner retries on the non-inflated map. If it still fails, it repicks the subgoal on the non-inflated map. If all attempts fail, it returns a crawl-forward action with the current heading and reduced speed.

This is an important engineering choice. It prevents the controller from deadlocking on a single bad frame and lets the next image provide a fresh replanning opportunity.

15 Collision Model and Simulator Semantics

15.1 Vehicle Footprint

Collision checking uses a simple 2D oriented rectangle footprint with default length 4.0 m and width 2.4 m. A five-point approximation is used:

- center,
- front-right,
- front-left,
- rear-right,
- rear-left.

If any of these sample points lies out of bounds or above-ground occupancy is present in the corresponding world voxel column, a collision is declared.

This is not exact polygon rasterization, but it is fast and usually sufficient at the chosen resolution.

15.2 Closed-Loop Rollout

At every simulator step:

1. render the current forward camera image from the ground-truth world;
2. run the occupancy model to obtain ego-frame predicted occupancy;
3. invoke the planner or RL policy;
4. update the car pose using the returned heading and step size;
5. check success, collision, out-of-bounds, stuckness, or timeout.

The crucial methodological point is that the controller never receives the ground-truth voxel world. Ground truth is used only for rendering and outcome scoring.

15.3 Success, Overshoot, and Failure Modes

A rollout terminates as

- **success**: distance to goal below tolerance,
- **collision**: footprint overlaps above-ground occupancy,
- **oob**: position leaves world bounds,
- **stuck**: insufficient progress over a recent window,
- **timeout**: maximum step count reached.

A notable nuance is the *overshoot success* heuristic. The simulator tracks the minimum goal distance achieved. If the vehicle once came within an expanded tolerance and is now moving away, the episode is still counted as success. This avoids penalizing a controller that effectively reached the target but crossed the success circle between discrete integration steps.

16 Reinforcement Learning Formulation

16.1 Training vs Deployment Distinction

The RL subsystem is intentionally split into two domains:

- **training domain**: a fast Gymnasium environment with oracle ego occupancy sampled directly from the voxel world,
- **deployment domain**: the full visual closed-loop simulator where the same policy acts on `OccNet` predictions.

This is a domain gap by design. It greatly accelerates policy training but means that a policy that performs well in training may degrade when fed perceptual predictions instead of perfect occupancy.

16.2 RL Observation

The observation is a flat vector of dimension

$$D_{\text{obs}} = D_x D_y + 5.$$

The first $D_x D_y$ entries are a BEV occupancy map flattened and encoded as

$$-1 = \text{free}, \qquad +1 = \text{occupied}.$$

The remaining features are

$$\text{goal}_x^{\text{norm}}, \quad \text{goal}_y^{\text{norm}}, \quad d_{\text{goal}}^{\text{norm}}, \quad a_{t-1}^{\text{turn}}, \quad a_{t-1}^{\text{speed}}.$$

If the world goal in ego coordinates is (g_x, g_y), then the normalized components are approximately

$$\text{goal}_x^{\text{norm}} = \text{clip}\left(\frac{g_x}{D_x\Delta}, -1, 1\right), \qquad \text{goal}_y^{\text{norm}} = \text{clip}\left(\frac{g_y}{(D_y\Delta)/2}, -1, 1\right),$$

$$d_{\text{goal}}^{\text{norm}} = \text{clip}\left(\frac{\|\mathbf{g} - \mathbf{p}\|_2}{d_{\max}}, 0, 1\right).$$

16.3 Action Space

The action is continuous:

$$\mathbf{a}_t = (u_t^{\text{turn}}, u_t^{\text{speed}}) \in [-1, 1]^2.$$

The turn command rotates the current heading by at most a fixed angular increment:

$$\Delta\theta_t = u_t^{\text{turn}}\, \theta_{\max}.$$

The speed command is mapped to a speed fraction between a minimum and one:

$$\alpha_t = \alpha_{\min} + \frac{u_t^{\text{speed}} + 1}{2}(1 - \alpha_{\min}), \qquad s_t = \alpha_t s_{\max}.$$

This makes the policy's action semantics nearly identical to the planner's output interface, which is why the `RLPolicyAdapter` is so thin.

16.4 Reward Function

Let d_t be distance to goal at step t. Define progress

$$\Delta d_t = d_{t-1} - d_t.$$

Then the reward is of the form

$$r_t = w_p \Delta d_t - w_s - w_{\text{turn}}|u_t^{\text{turn}}| - w_{\text{slow}}(1 - \alpha_t) + r_{\text{terminal}}.$$

Terminal adjustments are

$$r_{\text{terminal}} = \begin{cases} +R_{\text{succ}}, & \text{success}, \\ -R_{\text{col}}, & \text{collision}, \\ -R_{\text{oob}}, & \text{out of bounds}, \\ -R_{\text{to}}, & \text{timeout}, \\ 0, & \text{otherwise}. \end{cases}$$

This is a standard dense-progress shaping objective with penalties for time and aggressive control. It encourages shortest-feasible forward progress toward the exit.

16.5 PPO Integration

Training uses `stable_baselines3.PPO` with an MLP policy. Multiple vectorized environments are supported via either `DummyVecEnv` or `SubprocVecEnv`. The environment wrapper records monitor CSVs and the custom callback stores per-episode metrics, rolling plots, and summary statistics.

16.6 RL Adapter in Full Closed Loop

The full closed-loop RL script does not retrain or redesign the policy. It wraps a trained PPO model with `RLPolicyAdapter`, which:

1. constructs the same flat observation from predicted occupancy,
2. calls `model.predict`,
3. maps the output action to heading and step size,
4. packages the result as a `PlannerResult`.

This interface symmetry is elegant: both A* and PPO ultimately produce the same control object expected by the simulator.

17 Training Artifacts and Diagnostics for RL

The RL artifact module records one JSON row per completed episode. It computes rolling means of reward, episode length, success rate, collision rate, out-of-bounds rate, timeout rate, and final distance to goal.

This is a useful complement to raw PPO logs because it tracks task-level outcomes directly rather than only SB3 optimizer diagnostics.

A technical nuance is that the metrics code includes the label `stuck` among possible outcomes, even though the fast RL environment itself does not explicitly emit `stuck`. That label becomes relevant in the full visual simulator but not necessarily in the oracle training environment.

18 Kalman Filtering and Extended Kalman Filtering

Although not central to the occupancy-planning pipeline, the estimation module is mathematically coherent and worth documenting.

18.1 Linear Kalman Filter

The linear filter maintains Gaussian state

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, P)$$

with linear dynamics

$$\mathbf{x}_{t+1} = F\mathbf{x}_t + B\mathbf{u}_t + \mathbf{w}_t, \qquad \mathbf{w}_t \sim \mathcal{N}(0, Q),$$

and observation model

$$\mathbf{z}_t = H\mathbf{x}_t + \mathbf{v}_t, \qquad \mathbf{v}_t \sim \mathcal{N}(0, R).$$

Prediction is

$$\boldsymbol{\mu}^- = F\boldsymbol{\mu} + B\mathbf{u}, \qquad P^- = FPF^\top + Q.$$

Update is

$$\mathbf{y} = \mathbf{z} - H\boldsymbol{\mu}^-, \qquad S = HP^-H^\top + R, \qquad K = P^-H^\top S^{-1}.$$

Then

$$\boldsymbol{\mu}^+ = \boldsymbol{\mu}^- + K\mathbf{y}.$$

The code uses the Joseph covariance form:

$$P^+ = (I - KH)P^-(I - KH)^\top + KRK^\top,$$

which is numerically safer than the simpler algebraic equivalent.

18.2 Vehicle EKF

The EKF state is

$$\mathbf{x} = [x, y, \theta, v]^\top.$$

Controls are yaw rate ω and longitudinal acceleration a. The motion model is the standard unicycle-style discretization:

$$\begin{aligned} x' &= x + v\Delta t \cos\theta, \\ y' &= y + v\Delta t \sin\theta, \\ \theta' &= \theta + \omega\Delta t, \\ v' &= v + a\Delta t. \end{aligned}$$

The Jacobian with respect to the state is

$$F = \begin{bmatrix} 1 & 0 & -v\Delta t \sin\theta & \Delta t \cos\theta \\ 0 & 1 & v\Delta t \cos\theta & \Delta t \sin\theta \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The measurement model can include any subset of position, heading, and speed. Heading residuals are angle-wrapped into $[-\pi, \pi)$.

This is a clean educational EKF implementation and fits naturally with the project's synthetic nature.

19 Software Interfaces for Model Resolution and UI

19.1 Hugging Face Model Resolution

The hub module abstracts over local-path loading and remote Hub downloads. The design principle is simple: if the requested path exists locally, use it; otherwise treat it as a Hub filename and resolve it from a configured repository.

This makes model loading compositional and keeps the UI code independent of storage location.

19.2 Gradio UI Architecture

The Gradio application is split into two conceptual tabs:

1. a **random generator** tab for procedural data inspection and sample export,
2. a **closed-loop demo** tab for one-shot rollout experiments using either A* or PPO.

The UI is not just cosmetic. It exposes the project's main abstractions directly:

- camera geometry,
- world/trajectory randomness,
- rendered data products,
- model source selection,
- controller substitution.

20 Design Trade-offs and Technical Assessment

20.1 Strengths

1. **End-to-end coherence.** The project defines a full synthetic loop from world generation to deployment evaluation.
2. **Clear geometry.** Coordinate frames and transformations are unusually consistent and well documented.
3. **Efficient preprocessing.** Memmapped arrays are a practical choice for large synthetic datasets.
4. **Planner realism.** The A* stack includes several nontrivial engineering fixes: blind-spot pessimism, obstacle inflation, world-boundary safety, and goal slowdown.
5. **RL deployment realism.** Training on oracle occupancy but deploying on predicted occupancy creates a meaningful robustness test.
6. **Diagnostics.** Visualizations and artifact writing are strong relative to project size.

20.2 Limitations

1. **Perception model simplicity.** The image-to-BEV lift is implicit and may limit accuracy or extrapolation.
2. **No uncertainty modeling.** Occupancy is binary after thresholding; planner and RL policy do not exploit confidence.
3. **Single-frame control.** No temporal fusion is used for perception or planning.

4. **Synthetic-to-real gap.** The renderer is stylized, so transfer beyond this simulated domain would be limited without adaptation.
5. **Approximate collision footprint.** Five-point sampling is efficient but not exact.
6. **Oracle-RL mismatch.** RL training may overfit to perfect occupancy statistics rather than learned predictions.

20.3 Implementation-Level Nuances

Several details in the code deserve explicit mention:

- The default rendering height in `RenderConfig` is 135, while many scripts use 136 as a fallback. This is a small but real consistency hazard.
- The planner works in a BEV reduction of the 3D occupancy tensor, so vertical structure only matters insofar as it creates any above-ground occupancy.
- The static FOV mask is a major simplification that depends on fixed camera geometry; changing camera parameters across samples would invalidate this design.
- The close-range blind-spot workaround is heuristic. It is practical, but it is not a fully principled partially observable planner.

21 Mathematical Summary of the Full Pipeline

A concise formalization of the whole system is as follows.

21.1 World and Trajectory Sampling

Sample world parameters ω and trajectory parameters τ, then construct

$$\mathcal{V} = \mathcal{G}(\omega, \tau), \qquad \Gamma = \{\mathbf{p}_t\}_{t=0}^{T-1}.$$

21.2 Synthetic Observation Model

At simulator state $(\mathbf{x}_t, \mathbf{h}_t)$, render image

$$\mathbf{I}_t = \mathcal{R}(\mathcal{V}, \mathbf{x}_t, \mathbf{h}_t, \text{camera config}).$$

21.3 Ground-Truth Learning Target

Compute ego occupancy crop

$$\mathbf{Y}_t = \mathcal{E}(\mathcal{V}, \mathbf{x}_t, \mathbf{h}_t).$$

Apply static visibility mask M in training.

21.4 Perception

Predict logits

$$\mathbf{Z}_t = f_\theta(\mathbf{I}_t),$$

train by minimizing

$$\mathcal{L}(\theta) = \frac{1}{B \sum M} \sum_{b,i,j,k} M_{ijk}\, \ell(Z_{bijk}, Y_{bijk}).$$

21.5 Closed-Loop Control

Threshold occupancy:

$$\widehat{\mathbf{Y}}_t = \mathbf{1}\{\mathbf{Z}_t > 0\}.$$

Then either

$$\mathbf{u}_t = \pi_{\mathrm{A}^*}(\widehat{\mathbf{Y}}_t, \mathbf{x}_t, \mathbf{h}_t, \mathbf{g}),$$

or

$$\mathbf{u}_t = \pi_{\mathrm{RL}}(\widehat{\mathbf{Y}}_t, \mathbf{x}_t, \mathbf{h}_t, \mathbf{g}, \mathbf{u}_{t-1}).$$

Integrate the simulator:

$$(\mathbf{x}_{t+1}, \mathbf{h}_{t+1}) = \mathcal{S}(\mathbf{x}_t, \mathbf{h}_t, \mathbf{u}_t).$$

Evaluate against ground-truth termination rules.

22 Research Interpretation

From a research perspective, the repository can be read as testing the following proposition:

> Given a fixed monocular viewpoint and a procedurally generated but structured world distribution, a simple CNN can learn a near-field ego occupancy representation that is useful enough to support local search or learned control in closed loop.

The project answers that proposition by creating a synthetic dataset whose ground truth is geometrically exact, a perception model whose outputs are directly aligned with the planner's state representation, and a rollout engine that measures actual task completion rather than only reconstruction accuracy.

In that sense, the codebase is more than a simulator and more than a vision model. It is a compact experimental framework for comparing representation quality through control performance.

23 Potential Extensions

Natural technical extensions include:

- replacing the implicit BEV lift with explicit camera-aware lifting or transformer-based view projection;
- predicting occupancy probabilities and passing them as costs rather than thresholded obstacles;

- incorporating temporal memory or recurrent state estimation;
- training RL directly on noisy predicted occupancy rather than oracle occupancy;
- replacing local A* with differentiable planning, MPC, or kinodynamic search;
- introducing richer vehicle dynamics and continuous-time collision checking;
- adding domain randomization or improved rendering fidelity for transfer experiments.

24 Conclusion

The repository is a technically coherent synthetic autonomous-driving stack centered on monocular occupancy prediction. Its main contributions are not raw algorithmic novelty but the clarity with which the pipeline is built and the care taken to make the representation, training loss, planner, simulator, and evaluation scenarios all agree on geometry.

The world generator creates structured but diverse road scenes; the renderer provides first-person observations; the ego-grid machinery defines a stable supervised target; the perception model learns to predict that target; the planner and PPO controller consume that target in closed loop; and the simulator measures whether the resulting behavior actually reaches goals safely.

That combination makes the project useful both as a proof-of-concept research scaffold and as a pedagogical codebase for synthetic perception-and-control systems.

A Appendix: Module-by-Module Technical Notes

A.1 Top-Level Entry Points

The root scripts correspond to distinct pipeline phases:

- `generate.py`: procedural data generation,
- `preprocess.py`: memmap construction,
- `train.py`: occupancy-model optimization,
- `run_closed_loop.py`: A*-based deployment evaluation,
- `train_rl.py`: PPO training in oracle occupancy,
- `eval_rl.py`: PPO evaluation in oracle occupancy,
- `run_closed_loop_rl.py`: PPO deployment over OccNet predictions,
- `app.py`: Gradio launch,
- `scripts/upload_models_to_hf.py`: model artifact publication.

A.2 Package Export Strategy

The top-level package `voxel_car.__init__.py` re-exports the most commonly used functions and classes so callers can write concise imports. This is an API ergonomics choice rather than a mathematical one, but it improves the usability of the package as a research toolkit.

A.3 Scenario Difficulty Ladder

The named scenario presets are a difficulty ladder:

- `easy`: close to training distribution,
- `winding`: narrower roads, more turns,
- `tall_obstacles`: taller terrain structures,
- `narrow`: tight roads and bumpier terrain.

This is a good experimental design pattern because it permits controlled evaluation of generalization under structured distribution shift.

A.4 Why the RL Environment Is Fast

The oracle-BEV environment avoids two expensive operations:

1. camera rendering,
2. neural network inference.

Instead, it computes the ego occupancy tensor directly from the voxel world and current pose. This changes the observation distribution but preserves the action semantics and much of the local geometry.

B Appendix: Selected Equations in Code-Adjacent Form

For reference, the most important mathematical relations implemented in the repository are:

$$
\begin{aligned}
\text{Vehicle right vector:} \quad & \mathbf{r} = (h_y, -h_x), \\
\text{World to ego:} \quad & e_x = (x_w - x_c)h_x + (y_w - y_c)h_y, \\
& e_y = (x_w - x_c)h_y - (y_w - y_c)h_x, \\
\text{Camera heading:} \quad & \mathbf{h}_{\text{cam}} = \cos\psi\, \mathbf{h} + \sin\psi\, \mathbf{r}, \\
\text{Vertical FOV:} \quad & \varphi_v = 2\arctan\left(\frac{H}{W}\tan\frac{\varphi_h}{2}\right), \\
\text{Masked BCE:} \quad & \mathcal{L} = \frac{\sum M\,\ell(Z,Y)}{B\sum M + \varepsilon}, \\
\text{BEV collapse:} \quad & \mathcal{B}(i,j) = \bigvee_{k\geq 1} \widehat{Y}(i,j,k), \\
\text{Soft cost:} \quad & C_{\text{soft}} = we^{-d}, \\
\text{Goal slowdown:} \quad & s_{\text{eff}} = s\max\left(\rho_{\min}, \frac{d}{R}\right), \\
\text{RL speed map:} \quad & \alpha = \alpha_{\min} + \frac{u+1}{2}(1-\alpha_{\min}).
\end{aligned}
$$